

Examen : Programmation Parallèle - MPI Avancé

Durée : 1h45

20 décembre 2024

L'accès au support de cours et aux ressources informatiques est autorisé. Toutes les réponses doivent être justifiées. La qualité de la rédaction est prise en compte dans la notation. A noter que l'utilisation des agents conversationnels (type ChatGPT, LeChat, Claude, etc.) doit se faire intelligemment et avec parcimonie. Les réponses sont connues d'avance par le correcteur de cet examen.

1 Parallélisation d'un code de simulation numérique

N.B. : Des questions de cours sont dispersées tout au long de l'exercice. Elles sont repérées par un astérisque () après le numéro de la question.*

1.1 Modélisation numérique (10')

Un client d'un grand groupe industriel utilise un code séquentiel permettant de suivre l'évolution de la distribution de chaleur sur une plaque de métal soumise à une source de chaleur de chaque côté de cette plaque. La figure 1.1 (A) illustre cette plaque dans des conditions initiales et pour laquelle les bords droite et bas sont soumis à une température élevée (en rouge). La figure 1.1 (B) présente cette plaque à l'instant final. La température de la plaque a augmenté sur le voisinage des bords collés à la source de chaleur. Pour ce problème, le client considère une température initiale de la plaque à 0 degré avec une présence d'une source de chaleur de température constante égale à 100 tout autour de la plaque. La modélisation de ce problème est régit par l'équation de Laplace suivante :

$$\Delta U = 0$$

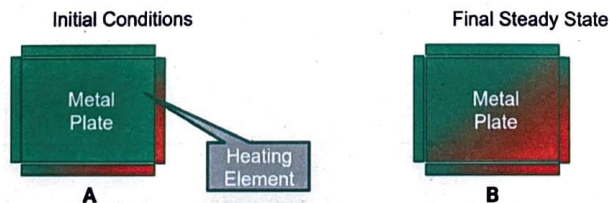


Figure 1: Illustration de la plaque de métal dans la condition initiale (A) et l'instant final (B) de la simulation. La couleur rouge représente une température plus élevée.

Au sein de la simulation, la plaque est représentée par un tableau de deux dimensions (appelé `Temperature_previous` dans le code source). Cette plaque est alors discrétisée sous forme de mailles pour lesquelles chacune contient une température notée $u_{i,j} = u(x_i, y_j)$. La résolution de l'équation de Laplace s'effectue suivant un processus itératif. A l'itération

$k + 1$, la température u à la maille (i, j) peut être calculée en utilisant les valeurs à l'itération k de ses voisins à l'aide de la formule suivante :

$$u^{k+1}(x_i, y_j) = \frac{1}{4}(u^k(x_{i+1}, y_j) + u^k(x_{i-1}, y_j) + u^k(x_i, y_{j+1}) + u^k(x_i, y_{j-1}))$$

La figure 2 illustre la position des températures voisines ainsi que les notations introduites dans le texte.

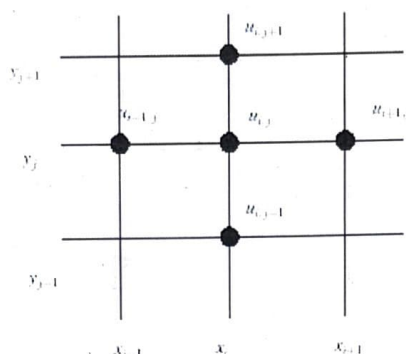


Figure 2: Illustration des positions des valeurs de température sur la grille

Le coeur de la simulation est défini par la boucle `while` suivante :

```

1 while ( worst_dt > TEMP_TOLERANCE ) {
2
3     for(i = 1; i <= HEIGHT; i++) {
4         for(j = 1; j <= WIDTH; j++) {
5             Temperature[i][j] = 0.25 * (Temperature_previous[i+1][j] +
6                                     Temperature_previous[i-1][j] +
7                                     Temperature_previous[i][j+1] +
8                                     Temperature_previous[i][j-1]);
9         }
10    }
11
12    worst_dt = 0.0;
13
14    for(i = 1; i <= HEIGHT; i++){
15        for(j = 1; j <= WIDTH; j++){
16            worst_dt = fmax( fabs(Temperature[i][j]-Temperature_previous[i][j]),
17                            worst_dt);
18            Temperature_previous[i][j] = Temperature[i][j];
19        }
20    }
21
22    iteration++;
23 }

```

- La première série de boucles `for` imbriquées permet de calculer $u_{i,j}^{k+1}$ à partir des températures précédentes des voisins, comme montré dans la formule précédente.
- La deuxième série de boucles `for` imbriquées calcule le changement maximal de température entre l'itération k et $k + 1$ sur l'ensemble des mailles (i, j) . Cette valeur est sauvee dans la variable `worst_dt`. Si ce changement est plus petit qu'un seuil de tolérance (défini par la variable `TEMP_TOLERANCE`), alors on considère que la simulation a convergé et le programme s'arrête normalement. On profite également de cette boucle pour mettre à jour la température calculée à l'itération $k + 1$ dans le tableau `Temperature_previous` pour la prochaine itération.

Problématique : le client nous demande de mettre en place une version parallèle MPI. Pour des soucis de confidentialité, ce code ne pourra tourner que sur leur propre ordinateur uniquement.

1.2 Décomposition de domaine (40')

Le problème étant défini sur une grille cartésienne 2D, nous allons tout d'abord effectuer une décomposition de domaine de cette grille à l'aide des sous-programmes MPI. Plus particulièrement, nous allons mettre en place une topologie en appelant les bons sous-programmes MPI.

Question 1* : Quel est le principe des topologies MPI et quelle est leur utilité sur ce type de problème ?

Pour simplifier la parallélisation, le client souhaite réaliser un découpage du domaine en bandes horizontales (donc un découpage seulement par rapport à la seconde dimension).

Question 2 : A partir des variables données dans le code ci-dessous, définissez une topologie cartésienne avec le communicateur associé en respectant le découpage voulu par le client.

```
1 MPI_Init(&argc, &argv);
2
3 int ndims = 2;
4 int nproc_per_dim[2] = {0, 0};
5 int periods_per_dim[2] = {0, 0};
6 int rang, nproc;
7
8 MPI_Comm_rank(MPI_COMM_WORLD, &(rang));
9 MPI_Comm_size(MPI_COMM_WORLD, &(nproc));
```

Question 3 : Le sous-programme `MPI_Cart_shift` permet de récupérer le rang des processus voisins dans une direction donnée (en x : 0, en y : 1). En s'aidant de la documentation ou du manuel disponible à la fin de l'énoncé (page 5), faites appel à ce sous-programme pour récupérer les processus voisins **sud** et **nord**. Vous pourrez utiliser le tableau `voisins` défini ci-dessous pour stocker le rang des voisins.

```
1 int voisins[2] = {MPI_PROC_NULL, MPI_PROC_NULL};
```

Question 4 : Maintenant que nous avons mis en place la topologie, un calcul du nombre de lignes des tableaux `Temperature_previous` et `Temperature` est effectué par rapport aux coordonnées des processus dans la topologie¹. On pose la variable `myrows` contenant le nombre de lignes pour chaque processus et la variable `ncols` pour le nombre de colonnes (similaire pour chaque processus). En ajoutant les mailles fantômes pour chaque processus, la taille des tableaux sont de `myrows+2` lignes et `ncols+2`. Rappelez quel est l'intérêt des mailles fantômes et que doivent-elles contenir ?

1.3 Communications (20')

Pour calculer correctement l'équation à chaque itération, il est nécessaire d'effectuer des communications avec les processus voisins. Le client souhaiterait passer par communications RMA plutôt que point-à-point.

Question 5* : Quel est le principe des communications *one-sided* ? Quelles sont les différences avec des communications point-à-point ?

Question 6 : Pour calculer correctement l'équation à chaque itération, il est nécessaire d'effectuer des communications avec les processus voisins. Avec le découpage en bandes horizontales, un processus doit communiquer avec ses processus voisins **nord** et **sud** respectivement. Nous allons mettre en place des communications *one-sided* avec le mécanisme de synchronisation minimal `pscw`. Réalisez les communications avec les voisins **nord** et **sud** en (i) créant (et en libérant) la fenêtre mémoire, (ii) en

¹Le nombre de colonne est toujours celui initial car le découpage a lieu seulement dans la seconde dimension.

implémentant le mécanisme *pscw* et (iii) en écrivant les données dans les processus voisins. On pourra s'aider du tableau *voisins* défini en question 3 pour créer le groupe MPI constitué des processus voisins.

1.4 Critère d'arrêt de la simulation (10')

L'arrêt de la simulation se repose sur une condition de la variable *worst_dt* par rapport à un seuil de tolérance (voir section 1.1). Avec le découpage en sous-domaines MPI, cette condition doit absolument être modifiée. En effet, il est nécessaire que chaque processus calcule cette condition et qu'une opération de réduction ait lieu pour calculer l'écart maximal de température entre deux itérations sur l'ensemble de la plaque de métal. Le sous-programme *MPI_Allreduce* doit alors être mis en place.

Question 7 : Le client se demande s'il serait judicieux d'utiliser la version non-bloquante de cette réduction pour gagner en performance. Il nous envoie alors un benchmark réalisé sur leur ordinateur pour ce sous-programme. Que peut-on répondre au client en analysant le benchmark ci-dessous ? Ecrivez alors l'appel au sous-programme le plus adapté pour la situation du client.

```

1 #-----
2 # Benchmarking Iallreduce
3 # #processes = 32
4 #-----

```

	#bytes	#repetitions	t_ovrl [usec]	t_pure [usec]	t_CPU [usec]	overlap [%]
5					0.23	0.00
6	0	1000	0.57	0.20	18.21	13.53
7	4	1000	34.02	18.28	18.30	13.44
8	8	1000	34.20	18.37	18.49	13.57
9	16	1000	34.50	18.52	18.74	13.89
10	32	1000	34.95	18.83	19.02	12.50
11	64	1000	35.76	19.13	31.31	23.25
12	128	1000	55.41	31.40	32.47	21.58
13	256	1000	57.95	32.49	34.61	20.30
14	512	1000	62.21	34.63	40.49	18.62
15	1024	1000	73.48	40.53	52.95	9.38
16	2048	1000	100.98	53.00	71.42	2.58
17	4096	1000	140.97	71.39	100.93	2.09
18	8192	1000	199.76	100.93	727.64	1.46
19	16384	1000	1443.36	726.32	783.63	1.34
20	32768	1000	1555.49	782.38	179.56	1.63
21	65536	640	356.88	180.26	311.88	3.58
22	131072	320	612.35	311.64	481.70	0.00
23	262144	160	963.05	480.98	9581.93	0.00
24	524288	80	19166.54	9578.30	10308.31	0.00
25	1048576	40	20617.79	10304.68	11710.33	0.00
26	2097152	20	23420.30	11706.60	14393.75	0.29
27	4194304	10	28740.24	14388.00		

1.5 Sauvegarde (25')

Une dernière étape consiste à mettre en place une sauvegarde parallèle du résultat de la simulation. Plus particulièrement, on souhaite écrire l'ensemble des valeurs de température sur la plaque de métal.

Question 8* : Quelle est l'approche conseillée lorsqu'on réalise des entrées/sorties sur un code parallèle et quelles sont celles déconseillées ? Expliquez pourquoi.

Pour simplifier la phase d'écriture, nous allons définir un type dérivé permettant de sélectionner facilement la partie du tableau à sauvegarder. En effet, n'ayant pas besoin de sauvegarder les mailles fantômes, nous aurons juste besoin des éléments intérieurs du tableau 2D (c'est-à-dire sans la première et dernière ligne et sans la première et dernière colonne).

Question 9* : Citez un sous-programme MPI permettant de sélectionner la partie voulue du tableau 2D. Expliquez ce sous-programme en détail et appliquez-le sur le code source.

Question 10 : Pour l'écriture, nous allons utiliser le sous-programme `MPI_File_write_at_all`. Décrivez l'ensemble des étapes avec le code correspondant pour appliquer ce sous-programme en n'oubliant pas d'ouvrir et de fermer le fichier.

```

1 MPI_Cart_shift
2
3 NAME
4     MPI_Cart_shift - Returns the shifted source and destination ranks,
                    given a shift direction and amount.
5
6 SYNTAX
7 C Syntax
8     #include <mpi.h>
9     int MPI_Cart_shift(MPI_Comm comm, int direction, int disp,
10                        int *source, int *dest)
11
12 comm
13     The communicator concerned.
14 direction
15     The index of the dimension in which the shift will be made.
16 disp
17     The number of units by which virtually move the topology.
18 source
19     The variable in which store the rank of the process that, given this shift,
                    would reach the calling process. In case no such process exists, which
                    may occur when the dimension traversed by the shift is not periodic,
                    MPI_PROC_NULL is stored instead.
20 dest
21     The rank of the process that, given this shift, would be reached by the
                    calling process. In case no such process exists, which may occur when
                    the dimension traversed by the shift is not periodic, MPI_PROC_NULL is
                    stored instead.
22
23 DESCRIPTION
24     MPI_Cart_shift virtually moves the cartesian topology of a communicator
                    (created with MPI_Cart_create) in the dimension specified. It permits
                    to find the two processes that would respectively reach, and be
                    reached by, the calling process with that shift. Shifting a cartesian
                    topology by 1 unit (the displacement) in a dimension therefore
                    allows a process to find its neighbours in that dimension. In case no
                    such neighbour exists, virtually located outside the boundaries of a
                    non periodic dimension for instance, MPI_PROC_NULL is given instead.
25
26 EXAMPLE
27     MPI_Comm new_communicator;
28     MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, reorder, &
                    new_communicator);
29     enum DIRECTIONS {DOWN, UP, LEFT, RIGHT};
30     char* neighbours_names[4] = {"down", "up", "left", "right"};
31     int neighbours_ranks[4];
32
33     MPI_Cart_shift(new_communicator, 0, 1, &neighbours_ranks[LEFT], &
                    neighbours_ranks[RIGHT]);
34     MPI_Cart_shift(new_communicator, 1, 1, &neighbours_ranks[DOWN], &
                    neighbours_ranks[UP]);

```